Step 1 — A Classifier That Sends Each Ticket to the Right Reply Chain — Solution

Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 1: every later step makes one part of this pipeline stronger — faster lookups, safer failures, a live reply, a record. Before any of that, prove the basic shape works: one chain decides the category, and a second chain, picked by that category, writes the reply. **If not:** when Step 3's fallback fires, you won't know whether the problem is the fallback or the routing under it.

After this step the project runs: `python -m langchainpro.main` classifies the 4 sample tickets and prints a reply for each.

The shared files

The same files as Project 1 — Doc01's config, errors and logger, and Project 1's prompt loader — with only the package name changed to `langchainpro`.

Story — `exceptions.py`, `config.py`, `utils/logger.py`, `utils/prompts.py`: settings load and fail loudly at startup, every file logs the same way, and prompts live in `.txt` files. **If not:** a missing key would surface deep inside a chain as a confusing error.

```
# src/langchainpro/exceptions.py
# why: one file for every named error in the project, so any module can
# raise or catch a specific problem instead of a bare Exception.

# from Doc01
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

```
# src/langchainpro/config.py
# how: os.getenv() is how we read env vars
import os
# why: @dataclass auto-builds __init__ for a typed holder
from dataclasses import dataclass
# how: reads .env and loads it into os.environ
from dotenv import load_dotenv
```

```
# why: a specific error type, not a bare Exception
from langchainpro.exceptions import MissingConfigError

@dataclass
class Config:
    # why: a typed object, not a plain dict — your editor autocompletes
    # .openai_api_key and .log_level, and catches a typo'd field name.
    openai_api_key: str
    log_level: str

def require_env(key: str) -> str:
    # why: one place that turns a missing/empty env var into a clear,
    # named error, instead of repeating the same if-check for every key.
    # when: called once per required key, inside load_config().
    value = os.getenv(key)
    if value is None or value == "":
        # how: both "never set" (None) and "set but blank" (empty string)
        # count as missing — an empty API key is never usable either way.
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config() -> Config:
    # why: the one function every other file calls to get settings — so a
    # missing key fails loudly here, at startup, not confusingly later.
    # when: call this once, at the top of main(), before anything else.
    # how: reads .env into os.environ; must run before os.getenv() below
    load_dotenv()
    # how: required — raises if missing
    api_key = require_env("OPENAI_API_KEY")
    # how: optional — "INFO" if unset
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)
```

```
# src/langchainpro/utils/logger.py
import logging
import os

def get_logger(name: str) -> logging.Logger:
    # why: gives every file in the project the same logging setup from one
    # place, instead of each file configuring handlers its own way.
    # when: logging.getLogger(name) always returns the same object for the
    # same name, so calling this again in the same run is safe and cheap.
    logger = logging.getLogger(name)
```

```
    if not logger.handlers:
        # how: only attach a handler the first time this name is
        # configured — without this guard, every line would print twice.
        level_name = os.getenv("LOG_LEVEL", "INFO")
        # how: falls back to INFO if the env value is invalid
        level = getattr(logging, level_name.upper(), logging.INFO)

        # how: prints to the console
        handler = logging.StreamHandler()
        handler.setLevel(level)
        # why: every line carries a timestamp/source/level, not just the
        # bare message
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        formatter = logging.Formatter(log_format)
        handler.setFormatter(formatter)

        logger.addHandler(handler)
        # how: the logger's own level must be at least this low, or the
        # handler never even sees the message
        logger.setLevel(level)

    return logger
```

```
# src/langchainpro/utils/prompts.py

# how: a path from the repo root — every command in this project is run
# from the repo root, the same rule the tests and scripts follow
PROMPTS_DIR = "src/langchainpro/prompts"

def load_prompt(name: str) -> str:
    # why: prompts live in .txt files, not inside Python strings — you
    # can reword an agent's behavior without touching its code.
    # when: called whenever an agent needs its system prompt.
    # how: e.g. "worker" -> "src/langchainpro/prompts/worker.txt"
    path = PROMPTS_DIR + "/" + name + ".txt"
    with open(path) as prompt_file:
        # how: strip() drops the blank line at the end of the file
        return prompt_file.read().strip()
```

Also create empty `__init__.py` files in `src/langchainpro/utils/` and `src/langchainpro/models/`.

> Back to this step · Hint 1 · Hint 2 · Solution

Solution — schema, chains, prompts and a service, each in its own file

Story — `models/llm.py`: one place builds the chat model, so the model choice lives in one line, never scattered (the rule from Doc05's Build Task). **If not:** every chain would build its own `ChatOpenAI` with its own settings.

```
# src/langchainpro/models/llm.py
from langchain_openai import ChatOpenAI

# why: one place to change the model for every chain in the project
MODEL_NAME = "gpt-4o-mini"

def get_model() -> ChatOpenAI:
    # why: every chain builds its model here, so they all share the same
    # model and settings
    # how: ChatOpenAI reads OPENAI_API_KEY from the environment, which
    # load_config() filled from .env (and checked) at startup
    return ChatOpenAI(model=MODEL_NAME, temperature=0)
```

Story — `schemas.py`: the classifier's answer is a contract: `category` can only be one of three values, `reason` explains the choice, and `sure` says whether the ticket could fit more than one category. `TicketRecord` is the result of one ticket — customer, ticket, category and reply — checked when it's built. Later steps only **add** fields to it, each with a default. **If not:** a typo like `"Billing"` would silently match no branch, and every caller would guess the keys of a loose dict.

```
# src/langchainpro/schemas.py
from typing import Literal

from pydantic import BaseModel

class TicketCategory(BaseModel):
    # why: the classifier's answer as a checked object, not free text —
    # only these 3 values can ever come back (Doc04's Literal)
    category: Literal["billing", "technical", "general"]
    # why: one short sentence, logged, so a wrong route can be explained
    reason: str
    # why: False when the ticket could fit more than one category — a
    # guess the rest of the pipeline should know about
```

```
    sure: bool

class TicketRecord(BaseModel):
    # why: one checked record per ticket — what a ticketing system would
    # store, count and route on (a machine reads this, not a person)
    customer_id: str
    ticket: str
    category: str
    reply: str
```

Story — `prompts/classifier.txt`: the rules for sorting, in plain text. `{ticket}` is the one slot the template fills. **If not:** rewording the classifier would mean editing Python.

`src/langchainpro/prompts/classifier.txt`

```
You sort customer support tickets into exactly one category:

- billing: charges, invoices, refunds, payments, plans
- technical: something in the product is broken or not working
- general: anything else, like questions about the company

Give a one-sentence reason. Set sure to false if the ticket could
fit more than one category.

Ticket: {ticket}
```

Story — the three reply prompts: one per category, named exactly like the category. Each has its own rules — billing never promises a refund, technical asks for a missing detail, general never invents a company fact. **If not:** one prompt trying to cover all three would do each one worse.

`src/langchainpro/prompts/billing.txt`

```
You are a billing support agent at SupportCo. Write a short, polite
reply to the customer, under 120 words. Never promise a refund: say
the billing team will check the charge within 2 business days.

Ticket: {ticket}
```

`src/langchainpro/prompts/technical.txt`

```
You are a technical support agent at SupportCo. Write a short, polite
reply to the customer, under 120 words. Suggest one or two simple
things to try, and ask for one missing detail (browser, device or
steps) if you need it.
```

```
Ticket: {ticket}
```

`src/langchainpro/prompts/general.txt`

```
You are a friendly support agent at SupportCo. Write a short, polite
reply to the customer, under 80 words. If you don't know a company
fact, say a team member will follow up. Never make one up.

Ticket: {ticket}
```

Story — **`chains/classifier.py`:** `build_classifier_chain()` returns the chain (Doc05's Build Task shape: a function that returns a `Runnable`). `classify_ticket()` runs it, catches `ValidationError` by name (Doc05's Intermediate exercise, `extract_person()`), and logs every decision with its reason, like Project 1's router. **If not:** one unreadable answer would crash the whole queue.

```
# src/langchainpro/chains/classifier.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable
from pydantic import ValidationError

from langchainpro.models.llm import get_model
from langchainpro.schemas import TicketCategory
from langchainpro.utils.logger import get_logger
from langchainpro.utils.prompts import load_prompt

def build_classifier_chain() -> Runnable:
    # why: prompt | structured model — Doc05's "shape during" method, so
    # the answer arrives as a checked TicketCategory, not text to parse
    prompt = ChatPromptTemplate.from_template(load_prompt("classifier"))
    model = get_model().with_structured_output(TicketCategory)
    chain = prompt | model
    return chain

def classify_ticket(ticket: str) -> TicketCategory:
    logger = get_logger(__name__)
    chain = build_classifier_chain()
    try:
        result = chain.invoke({"ticket": ticket})
    except ValidationError:
        # why: an answer that doesn't fit the shape must not crash the
        # pipeline — the safe default is "general, and not sure"
        logger.warning("Classifier answer did not fit TicketCategory")
        result = TicketCategory(
```

```
            category="general",
            reason="The classifier's answer could not be read.",
            sure=False,
        )
    # why: every routing decision is logged with its reason (Project 1)
    logger.info(
        f"category={result.category} sure={result.sure} "
        f"reason={result.reason}"
    )
    return result
```

Story — `chains/replies.py`: the branch. The category picks the prompt file, and the rest of the chain is the same for all three. **If not:** three nearly-identical functions, one per category.

```
# src/langchainpro/chains/replies.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable

from langchainpro.models.llm import get_model
from langchainpro.utils.prompts import load_prompt

def build_reply_chain(category: str) -> Runnable:
    # why: this is the branch — one reply chain per category, each with
    # its own prompt file (billing.txt, technical.txt, general.txt)
    # how: the category name IS the prompt file's name
    prompt = ChatPromptTemplate.from_template(load_prompt(category))
    chain = prompt | get_model() | StrOutputParser()

    return chain
```

Story — `services/pipeline.py`: the flow — classify, pick, reply — in one function, `handle_ticket()`, that prints the reply and returns a `TicketRecord`. The tests and `main.py` call the exact same code. **If not:** the flow would live inside `main.py`'s loop and nothing else could reuse it.

```
# src/langchainpro/services/pipeline.py
from langchainpro.chains.classifier import classify_ticket
from langchainpro.chains.replies import build_reply_chain
from langchainpro.schemas import TicketRecord

def handle_ticket(ticket: str, customer_id: str) -> TicketRecord:
    # how: classify -> pick the reply chain -> get the reply -> build
    # the record
    category = classify_ticket(ticket)
```

```
    chain = build_reply_chain(category.category)
    # how: every reply chain gets the same input dict; a prompt simply
    # ignores keys it doesn't use (Doc05's template edge cases)
    inputs = {"ticket": ticket, "customer_id": customer_id}
    reply = chain.invoke(inputs)
    print(f"Reply: {reply}")

    return TicketRecord(
        customer_id=customer_id,
        ticket=ticket,
        category=category.category,
        reply=reply,
    )
```

Story — `data/sample/tickets.json` **:** the 4 test tickets, including the unclear one. **If not:** the tickets would be retyped in `main.py` and in every test.

`data/sample/tickets.json`

```
[
  {
    "customer_id": "C-1001",
    "ticket": "I was charged twice for my Pro plan this month (INV-2201).",
    "expected": "billing"
  },
  {
    "customer_id": "C-1002",
    "ticket": "The export button spins forever and never downloads.",
    "expected": "technical"
  },
  {
    "customer_id": "C-1003",
    "ticket": "Do you have a referral program for other companies?",
    "expected": "general"
  },
  {
    "customer_id": "C-1004",
    "ticket": "My invoice and the dashboard numbers don't match. Bug?",
    "expected": "billing or technical"
  }
]
```

Story — `main.py` **:** config first, then every sample ticket through `handle_ticket()`; `print_record()` shows the result under each ticket. A bad key stops the app; a rate limit skips one ticket (Doc04's error table). **If not:** the first API error would end the run with a traceback.

```
# src/langchainpro/main.py
import json
import sys

import openai

from langchainpro.config import load_config
from langchainpro.exceptions import MissingConfigError
from langchainpro.schemas import TicketRecord
from langchainpro.services.pipeline import handle_ticket
from langchainpro.utils.logger import get_logger

# how: a path from the repo root — run every command from there
SAMPLES_PATH = "data/sample/tickets.json"

def load_samples() -> list[dict]:
    # why: the same 4 tickets every run, shared with the tests (Doc02
    # taught reading JSON)
    with open(SAMPLES_PATH) as samples_file:
        # how: read the file as text, then json.loads() it (Doc02)
        return json.loads(samples_file.read())

def print_record(record: TicketRecord) -> None:
    print(f"  category:      {record.category}")

def main() -> None:
    # why: config first — a missing OPENAI_API_KEY stops the app here
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    samples = load_samples()
    print(f"LangChainPro — {len(samples)} tickets in the queue.")

    for sample in samples:
        print()
        print(f"Ticket from {sample['customer_id']}: {sample['ticket']}")
        try:
            record = handle_ticket(sample["ticket"], sample["customer_id"])
        except openai.AuthenticationError:
            # why: a bad key is permanent — no retry fixes it, so exit
            logger.error("Authentication failed — check OPENAI_API_KEY.")
            print("Login failed — check OPENAI_API_KEY in your .env file.")
            sys.exit(1)
```

```
        except openai.RateLimitError:
            # how: the client already retried twice before this reached us
            print("We're busy right now — this ticket was skipped.")
            continue
        print_record(record)

if __name__ == "__main__":
    main()
```

Story — the tests: `test_schemas.py` needs no key and proves an unknown category is rejected. `test_live_pipeline.py` makes real calls and prints each category next to the one expected. **If not:** you'd only know the classifier works on the tickets you happened to try by hand.

```
# tests/unit/test_schemas.py
# Checks the Pydantic shapes accept good data and reject bad data.
# No API key needed. Run from the repo root:
#   python tests/unit/test_schemas.py
from pydantic import ValidationError

from langchainpro.schemas import TicketCategory, TicketRecord

def test_good_category() -> None:
    result = TicketCategory(
        category="billing", reason="A double charge.", sure=True
    )
    print(f"case 1 (billing)          -> {result.category}")

def test_unknown_category() -> None:
    # why: "shipping" is not one of the 3 categories — must be rejected
    try:
        TicketCategory(category="shipping", reason="?", sure=True)
        print("case 2 (shipping)         -> accepted (wrong!)")
    except ValidationError:
        print("case 2 (shipping)         -> ValidationError")

def test_record() -> None:
    # how: only the 4 fields every step has — later fields have defaults
    record = TicketRecord(
        customer_id="C-1001",
        ticket="I was charged twice this month.",
        category="billing",
        reply="Sorry about that — we'll refund the extra charge.",
    )
    print(f"case 3 (record)           -> {record.category}")

if __name__ == "__main__":
```

```
    test_good_category()
    test_unknown_category()
    test_record()
```

```
# tests/integration/test_live_pipeline.py
# Runs the classifier and the whole pipeline against the real API.
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python tests/integration/test_live_pipeline.py
import json

from langchainpro.chains.classifier import classify_ticket
from langchainpro.config import load_config
from langchainpro.services.pipeline import handle_ticket

def load_samples() -> list[dict]:
    with open("data/sample/tickets.json") as samples_file:
        # how: read the file as text, then json.loads() it (Doc02)
        return json.loads(samples_file.read())

def test_classification(samples: list[dict]) -> None:
    # case 1: every sample ticket gets the category we expect
    for sample in samples:
        result = classify_ticket(sample["ticket"])
        print(f"case 1 (expected {sample['expected']}) -> "
              f"{result.category}, sure={result.sure}")

def test_full_pipeline(samples: list[dict]) -> None:
    # case 2: the billing ticket goes all the way to a reply
    billing = samples[0]
    record = handle_ticket(billing["ticket"], billing["customer_id"])
    print(f"case 2 (billing)  -> {record.category}")

if __name__ == "__main__":
    load_config()
    samples = load_samples()
    test_classification(samples)
    test_full_pipeline(samples)
```

Also create empty `__init__.py` files in `src/langchainpro/chains/` and `src/langchainpro/services/`.

Run it:

```
python -m langchainpro.main
python tests/unit/test_schemas.py
```

```
python tests/integration/test_live_pipeline.py
```

`test_schemas.py` prints:

```
case 1 (billing)          -> billing
case 2 (shipping)         -> ValidationError
case 3 (record)           -> billing
```

Look closely at the unclear ticket (`C-1004`): which category did it get, and did the model say `sure=False` ? Nothing acts on `sure` yet — Step 4 does.